Case Study 1

Platform Foundation

"From Zero to API"

Labs 1-2 — Weeks 1-2

Course: Big Data and Intelligent Analytics
Instructor: Sri Krishnamurthy — QuantUniversity
Term: Spring 2026

Assigned: January 23, 2026
Due: February 6, 2026 (3:59 PM)

This case study builds the foundation for the entire PE Org-AI-R platform.

You will create a working API with data validation and persistence—the shell that all future case studies will extend. **Get this right, and everything else follows.**

Weight	8% of final grade
Format	Individual work
Submission	GitHub repository + README
Technologies	FastAPI, Pydantic, Docker, Snowflake, Redis, AWS S3
Lab Foundation	Lab 1 (Week 1) + Lab 2 (Week 2)

Contents

1 Business Context **4**
- 1.1 Why This Matters . . . 4
- 1.2 The PE Org-AI-R Framework . . . 4

2 Technical Requirements **6**
- 2.1 What You're Building . . . 6
- 2.2 Project Structure . . . 6

3 Data Model Specification **8**
- 3.1 Entity-Relationship Diagram . . . 8
- 3.2 Pydantic Model Specifications . . . 8
  - 3.2.1 Enumerations . . . 8
  - 3.2.2 Company Model . . . 9
  - 3.2.3 Assessment Model . . . 9
  - 3.2.4 Dimension Score Model . . . 10

4 API Endpoint Specification **12**
- 4.1 Required Endpoints . . . 12
- 4.2 Health Check Endpoint . . . 12
- 4.3 Pagination . . . 13

5 Database Schema (Snowflake) **14**
- 5.1 DDL Statements . . . 14
- 5.2 Seed Data . . . 15

6 Caching Strategy (Redis) **16**
- 6.1 What to Cache . . . 16
- 6.2 Cache Implementation . . . 16

7 Docker Configuration **18**
- 7.1 Dockerfile . . . 18
- 7.2 Docker Compose . . . 18

8 Evaluation Criteria **20**
- 8.1 Requirements Checklist . . . 20
- 8.2 Grading Rubric . . . 21

9 Submission Instructions **22**
- 9.1 What to Submit . . . 22
- 9.2 Submission Process . . . 22
- 9.3 Academic Integrity . . . 22

10 Resources **23**
- 10.1 Documentation . . . 23
- 10.2 Starter Repository . . . 23
- 10.3 Getting Help . . . 23

11 Looking Ahead **23**

1 Business Context

Business Scenario

You are a data engineer at a private equity firm.
Your firm is building an internal platform to assess the AI-readiness of portfolio companies and acquisition targets. The partners want a systematic way to evaluate whether companies are prepared to leverage AI for value creation.
The platform will eventually:

- Ingest SEC filings and extract AI-related evidence
- Score companies across 7 dimensions of AI-readiness
- Project financial impact (EBITDA improvements)
- Generate recommendations for improvement

Your task: Build the platform foundation—the API layer, data models, and persistence layer that all other components will depend on.

1.1 Why This Matters

Private equity firms are increasingly using AI-readiness as a factor in deal sourcing, due diligence, and value creation planning. A company with strong AI capabilities can achieve:

- **5–25% EBITDA improvement** through AI-driven operational efficiency
- **0.5–2.0x multiple expansion** at exit due to AI premium
- **Faster time-to-value** on operational improvements

But most PE firms assess AI-readiness through ad-hoc conversations and subjective judgment. Your platform will bring data-driven rigor to this process.

1.2 The PE Org-AI-R Framework

The platform implements the **PE Org-AI-R (Organizational AI-Readiness)** framework, which scores companies across seven dimensions:

Table 1: The Seven Dimensions of AI-Readiness

#	Dimension	Weight	What It Measures
D_1	Data Infrastructure	0.25	Quality, accessibility, and governance of data assets
D_2	AI Governance	0.20	Policies, ethics frameworks, compliance readiness
D_3	Technology Stack	0.15	Cloud infrastructure, ML tooling, API architecture
D_4	Talent & Skills	0.15	AI/ML talent density, retention, training programs
D_5	Leadership & Vision	0.10	Executive commitment, AI strategy, investment appetite
D_6	Use Case Portfolio	0.10	AI projects in production, pipeline, ROI tracking
D_7	Culture & Change	0.05	Innovation culture, change readiness, adoption rates

Each dimension is scored 0–100. The overall V^R (Value-Readiness) score is calculated as:

$$V^R = \bar{D}_w \times (1 - \lambda \times cv_D) \times \text{TalentRiskAdj} \tag{1}$$

Where:

- $\bar{D}_w$ = Weighted mean of dimension scores
- $\lambda = 0.25$ (non-compensatory penalty coefficient)
- cv_D = Coefficient of variation across dimensions
- TalentRiskAdj = Adjustment for key-person dependency

Hint

You don't need to implement the scoring formula in this case study. Focus on the data model that will *store* these scores. The scoring engine comes in Case Study 3.

2 Technical Requirements

2.1 What You're Building

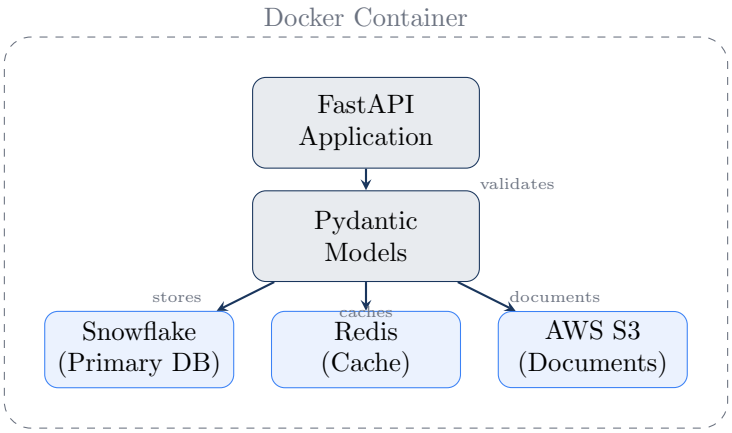

Figure 1: Case Study 1 Architecture

2.2 Project Structure

Your submission must follow this structure:

```
pe-org-air-platform/
|-- app/
|   |-- __init__.py
|   |-- main.py              # FastAPI application entry point
|   |-- config.py            # Configuration management
|   |-- models/
|   |   |-- __init__.py
|   |   |-- company.py       # Company Pydantic models
|   |   |-- assessment.py    # Assessment Pydantic models
|   |   |-- dimension.py     # Dimension score models
|   |-- routers/
|   |   |-- __init__.py
|   |   |-- companies.py     # Company CRUD endpoints
|   |   |-- assessments.py   # Assessment endpoints
|   |   |-- health.py        # Health check endpoint
|   |-- services/
|   |   |-- __init__.py
|   |   |-- snowflake.py     # Snowflake connection
|   |   |-- redis_cache.py   # Redis caching layer
|   |   |-- s3_storage.py    # S3 document storage
|   |-- database/
|       |-- __init__.py
|       |-- schema.sql       # Snowflake DDL
|-- tests/
|   |-- __init__.py
|   |-- test_models.py       # Model validation tests
|   |-- test_api.py          # API endpoint tests
|   |-- conftest.py          # Pytest fixtures
|-- docker/
```

```
|    |-- Dockerfile
|    |-- docker-compose.yml
|-- requirements.txt
|-- README.md
|-- .env.example
```

3 Data Model Specification

3.1 Entity-Relationship Diagram

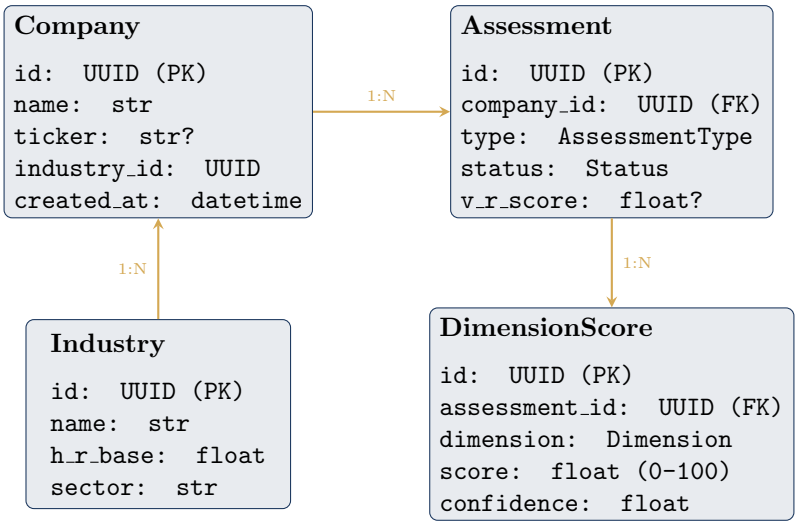

Figure 2: Core Entity Relationships

3.2 Pydantic Model Specifications

3.2.1 Enumerations

```python
from enum import Enum

class AssessmentType(str, Enum):
    SCREENING = "screening"            # Quick external assessment
    DUE_DILIGENCE = "due_diligence"    # Deep dive with internal access
    QUARTERLY = "quarterly"            # Regular portfolio monitoring
    EXIT_PREP = "exit_prep"            # Pre-exit assessment

class AssessmentStatus(str, Enum):
    DRAFT = "draft"
    IN_PROGRESS = "in_progress"
    SUBMITTED = "submitted"
    APPROVED = "approved"
    SUPERSEDED = "superseded"

class Dimension(str, Enum):
    DATA_INFRASTRUCTURE = "data_infrastructure"
    AI_GOVERNANCE = "ai_governance"
    TECHNOLOGY_STACK = "technology_stack"
    TALENT_SKILLS = "talent_skills"
    LEADERSHIP_VISION = "leadership_vision"
    USE_CASE_PORTFOLIO = "use_case_portfolio"
    CULTURE_CHANGE = "culture_change"
```

3.2.2 Company Model

Requirements

Company must include:

- UUID primary key (auto-generated)
- Name (required, 1-255 characters)
- Ticker symbol (optional, uppercase, 1-10 characters)
- Industry reference (UUID foreign key)
- Position factor (-1.0 to 1.0, default 0.0)
- Created/updated timestamps

```python
from pydantic import BaseModel, Field, field_validator, model_validator
from uuid import UUID, uuid4
from datetime import datetime, timezone
from typing import Optional

class CompanyBase(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    ticker: Optional[str] = Field(None, max_length=10)
    industry_id: UUID
    position_factor: float = Field(default=0.0, ge=-1.0, le=1.0)

    @field_validator('ticker')
    @classmethod
    def uppercase_ticker(cls, v: Optional[str]) -> Optional[str]:
        return v.upper() if v else None

class CompanyCreate(CompanyBase):
    pass

class CompanyResponse(CompanyBase):
    id: UUID
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True
```

3.2.3 Assessment Model

Requirements

Assessment must include:

- UUID primary key
- Company foreign key reference
- Assessment type (enum)
- Assessment date
- Status with state machine validation
- Optional V^R score (0-100, calculated later)

- Confidence interval (lower, upper bounds)
- Assessor information

```python
class AssessmentBase(BaseModel):
    company_id: UUID
    assessment_type: AssessmentType
    assessment_date: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
    primary_assessor: Optional[str] = None
    secondary_assessor: Optional[str] = None

class AssessmentCreate(AssessmentBase):
    pass

class AssessmentResponse(AssessmentBase):
    id: UUID
    status: AssessmentStatus = AssessmentStatus.DRAFT
    v_r_score: Optional[float] = Field(None, ge=0, le=100)
    confidence_lower: Optional[float] = Field(None, ge=0, le=100)
    confidence_upper: Optional[float] = Field(None, ge=0, le=100)
    created_at: datetime

    @model_validator(mode='after')
    def validate_confidence_interval(self) -> 'AssessmentResponse':
        if (self.confidence_upper is not None and
            self.confidence_lower is not None and
            self.confidence_upper < self.confidence_lower):
            raise ValueError('confidence_upper must be >= confidence_lower
                ')
        return self

    class Config:
        from_attributes = True
```

3.2.4 Dimension Score Model

Requirements

DimensionScore must include:

- UUID primary key
- Assessment foreign key reference
- Dimension type (enum)
- Score (0-100, required)
- Weight (0-1, dimension-specific default)
- Confidence level (0-1)
- Evidence count (how many pieces of evidence support this score)

```python
# Default weights per dimension
DIMENSION_WEIGHTS = {
    Dimension.DATA_INFRASTRUCTURE: 0.25,
```

```
    Dimension.AI_GOVERNANCE: 0.20,
    Dimension.TECHNOLOGY_STACK: 0.15,
    Dimension.TALENT_SKILLS: 0.15,
    Dimension.LEADERSHIP_VISION: 0.10,
    Dimension.USE_CASE_PORTFOLIO: 0.10,
    Dimension.CULTURE_CHANGE: 0.05,
}

class DimensionScoreBase(BaseModel):
    assessment_id: UUID
    dimension: Dimension
    score: float = Field(..., ge=0, le=100)
    weight: Optional[float] = Field(default=None, ge=0, le=1)
    confidence: float = Field(default=0.8, ge=0, le=1)
    evidence_count: int = Field(default=0, ge=0)

    @model_validator(mode='after')
    def set_default_weight(self) -> 'DimensionScoreBase':
        if self.weight is None:
            self.weight = DIMENSION_WEIGHTS.get(self.dimension, 0.1)
        return self

class DimensionScoreCreate(DimensionScoreBase):
    pass

class DimensionScoreResponse(DimensionScoreBase):
    id: UUID
    created_at: datetime

    class Config:
        from_attributes = True
```

4 API Endpoint Specification

4.1 Required Endpoints

Table 2: API Endpoints to Implement

Method	Endpoint	Description
Health		
GET	/health	Health check with dependency status
Companies		
POST	/api/v1/companies	Create a new company
GET	/api/v1/companies	List companies (with pagination)
GET	/api/v1/companies/{id}	Get company by ID
PUT	/api/v1/companies/{id}	Update company
DELETE	/api/v1/companies/{id}	Soft delete company
Assessments		
POST	/api/v1/assessments	Create new assessment
GET	/api/v1/assessments	List assessments (filterable)
GET	/api/v1/assessments/{id}	Get assessment with scores
PATCH	/api/v1/assessments/{id}/status	Update assessment status
Dimension Scores		
POST	/api/v1/assessments/{id}/scores	Add dimension scores
GET	/api/v1/assessments/{id}/scores	Get all dimension scores
PUT	/api/v1/scores/{id}	Update a dimension score

4.2 Health Check Endpoint

The health endpoint must return status of all dependencies:

```python
from fastapi import APIRouter, status
from pydantic import BaseModel
from typing import Dict
from datetime import datetime, timezone

router = APIRouter()

class HealthResponse(BaseModel):
    status: str
    timestamp: datetime
    version: str
    dependencies: Dict[str, str]

@router.get("/health", response_model=HealthResponse)
async def health_check():
    """
    Check health of all dependencies.
    Returns 200 if all healthy, 503 if any unhealthy.
```

```
    """
    dependencies = {
        "snowflake": await check_snowflake(),
        "redis": await check_redis(),
        "s3": await check_s3(),
    }

    all_healthy = all(v == "healthy" for v in dependencies.values())

    return HealthResponse(
        status="healthy" if all_healthy else "degraded",
        timestamp=datetime.now(timezone.utc),
        version="1.0.0",
        dependencies=dependencies
    )
```

4.3 Pagination

List endpoints must support pagination:

```
from pydantic import BaseModel
from typing import Generic, TypeVar, List

T = TypeVar('T')

class PaginatedResponse(BaseModel, Generic[T]):
    items: List[T]
    total: int
    page: int
    page_size: int
    total_pages: int

@router.get("/companies", response_model=PaginatedResponse[CompanyResponse
    ])
async def list_companies(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    industry_id: Optional[UUID] = None,
):
    """List companies with optional filtering."""
    # Implementation here
    pass
```

5 Database Schema (Snowflake)

5.1 DDL Statements

Create these tables in Snowflake:

```sql
-- Industries table
CREATE TABLE IF NOT EXISTS industries (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    sector VARCHAR(100) NOT NULL,
    h_r_base DECIMAL(5,2) CHECK (h_r_base BETWEEN 0 AND 100),
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Companies table
CREATE TABLE IF NOT EXISTS companies (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    ticker VARCHAR(10),
    industry_id VARCHAR(36) REFERENCES industries(id),
    position_factor DECIMAL(4,3) DEFAULT 0.0
        CHECK (position_factor BETWEEN -1.0 AND 1.0),
    is_deleted BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
    updated_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Assessments table
CREATE TABLE IF NOT EXISTS assessments (
    id VARCHAR(36) PRIMARY KEY,
    company_id VARCHAR(36) NOT NULL REFERENCES companies(id),
    assessment_type VARCHAR(20) NOT NULL
        CHECK (assessment_type IN
            ('screening', 'due_diligence', 'quarterly', 'exit_prep')),
    assessment_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'draft'
        CHECK (status IN
            ('draft', 'in_progress', 'submitted', 'approved', 'superseded'
                )),
    primary_assessor VARCHAR(255),
    secondary_assessor VARCHAR(255),
    v_r_score DECIMAL(5,2) CHECK (v_r_score BETWEEN 0 AND 100),
    confidence_lower DECIMAL(5,2),
    confidence_upper DECIMAL(5,2),
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Dimension scores table
CREATE TABLE IF NOT EXISTS dimension_scores (
    id VARCHAR(36) PRIMARY KEY,
    assessment_id VARCHAR(36) NOT NULL REFERENCES assessments(id),
    dimension VARCHAR(30) NOT NULL
        CHECK (dimension IN (
```

```sql
            'data_infrastructure', 'ai_governance', 'technology_stack',
            'talent_skills', 'leadership_vision', 'use_case_portfolio',
            'culture_change'
        )),
    score DECIMAL(5,2) NOT NULL CHECK (score BETWEEN 0 AND 100),
    weight DECIMAL(4,3) CHECK (weight BETWEEN 0 AND 1),
    confidence DECIMAL(4,3) DEFAULT 0.8 CHECK (confidence BETWEEN 0 AND 1),
    evidence_count INT DEFAULT 0,
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
    UNIQUE (assessment_id, dimension)
);

-- Indexes for common queries
CREATE INDEX IF NOT EXISTS idx_companies_industry
    ON companies(industry_id);
CREATE INDEX IF NOT EXISTS idx_assessments_company
    ON assessments(company_id);
CREATE INDEX IF NOT EXISTS idx_dimension_scores_assessment
    ON dimension_scores(assessment_id);
```

5.2 Seed Data

Include seed data for industries:

```sql
INSERT INTO industries (id, name, sector, h_r_base) VALUES
    ('550e8400-e29b-41d4-a716-446655440001', 'Manufacturing',
     'Industrials', 72),
    ('550e8400-e29b-41d4-a716-446655440002', 'Healthcare Services',
     'Healthcare', 78),
    ('550e8400-e29b-41d4-a716-446655440003', 'Business Services',
     'Services', 75),
    ('550e8400-e29b-41d4-a716-446655440004', 'Retail',
     'Consumer', 70),
    ('550e8400-e29b-41d4-a716-446655440005', 'Financial Services',
     'Financial', 80);
```

6 Caching Strategy (Redis)

6.1 What to Cache

Table 3: Caching Strategy

Data	TTL	Rationale
Company by ID	5 minutes	Frequently accessed, rarely changes
Industry list	1 hour	Static reference data
Assessment by ID	2 minutes	May be updated during work
Dimension weights	24 hours	Configuration, rarely changes

6.2 Cache Implementation

```python
import redis
import json
from typing import Optional, TypeVar, Type
from pydantic import BaseModel

T = TypeVar('T', bound=BaseModel)

class RedisCache:
    def __init__(self, host: str, port: int, db: int = 0):
        self.client = redis.Redis(host=host, port=port, db=db)

    def get(self, key: str, model: Type[T]) -> Optional[T]:
        """Get cached item, deserialize to Pydantic model."""
        data = self.client.get(key)
        if data:
            return model.model_validate_json(data)
        return None

    def set(self, key: str, value: BaseModel, ttl_seconds: int) -> None:
        """Cache Pydantic model with TTL."""
        self.client.setex(
            key,
            ttl_seconds,
            value.model_dump_json()
        )

    def delete(self, key: str) -> None:
        """Invalidate cache entry."""
        self.client.delete(key)

    def delete_pattern(self, pattern: str) -> None:
        """Invalidate all keys matching pattern."""
        for key in self.client.scan_iter(match=pattern):
            self.client.delete(key)

# Usage example
cache = RedisCache(host="localhost", port=6379)
```

```python

async def get_company(company_id: UUID) -> CompanyResponse:
    cache_key = f"company:{company_id}"

    # Try cache first
    cached = cache.get(cache_key, CompanyResponse)
    if cached:
        return cached

    # Fetch from database
    company = await db.fetch_company(company_id)

    # Cache for 5 minutes
    cache.set(cache_key, company, ttl_seconds=300)

    return company
```

Warning

Cache Invalidation: When updating or deleting a company, remember to invalidate the cache! This is a common source of bugs.

7 Docker Configuration

7.1 Dockerfile

```
# Dockerfile for PE Org-AI-R Platform
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app/ ./app/

# Create non-root user
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Run application
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7.2 Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - SNOWFLAKE_ACCOUNT=${SNOWFLAKE_ACCOUNT}
      - SNOWFLAKE_USER=${SNOWFLAKE_USER}
      - SNOWFLAKE_PASSWORD=${SNOWFLAKE_PASSWORD}
      - SNOWFLAKE_DATABASE=${SNOWFLAKE_DATABASE}
      - SNOWFLAKE_SCHEMA=${SNOWFLAKE_SCHEMA}
      - SNOWFLAKE_WAREHOUSE=${SNOWFLAKE_WAREHOUSE}
      - REDIS_HOST=redis
```

```
      - REDIS_PORT=6379
      - AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
      - AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
      - S3_BUCKET=${S3_BUCKET}
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    restart: unless-stopped

volumes:
  redis_data:
```

8 Evaluation Criteria

8.1 Requirements Checklist

Your submission must include all of the following components:

Requirements

All requirements must be completed for full credit (100 points)

Data Models (25 points)
- ✓ All Pydantic models implemented (Company, Assessment, DimensionScore, Industry)
- ✓ Proper validation rules (field constraints, type checking)
- ✓ Enum types for AssessmentType, AssessmentStatus, Dimension

API Endpoints (30 points)
- ✓ All 12 REST endpoints functional
- ✓ Health check returns dependency status
- ✓ Pagination on list endpoints
- ✓ Proper error handling with meaningful messages

Data Persistence (20 points)
- ✓ Snowflake schema created with all tables
- ✓ Redis caching implemented for companies and industries
- ✓ CRUD operations working end-to-end

Infrastructure (15 points)
- ✓ Docker builds and runs successfully
- ✓ docker-compose orchestrates all services
- ✓ Environment configuration via .env files

Quality & Documentation (10 points)
- ✓ Tests pass (provided test suite + your additions)
- ✓ README with setup instructions
- ✓ API documentation via OpenAPI/Swagger

8.2 Grading Rubric

Table 4: Detailed Grading Rubric

Component	Points
Data Models (25 points)	
Pydantic models (all 4 entities)	15
Validation rules & enums	10
API Endpoints (30 points)	
Company endpoints (5 endpoints)	10
Assessment endpoints (4 endpoints)	8
Dimension score endpoints (3 endpoints)	6
Health check endpoint	3
Pagination implementation	3
Data Persistence (20 points)	
Snowflake schema & connection	10
Redis caching layer	7
Cache invalidation on updates	3
Infrastructure (15 points)	
Dockerfile (builds, runs)	8
docker-compose configuration	5
Environment configuration	2
Quality & Documentation (10 points)	
Test coverage	4
README completeness	3
OpenAPI/Swagger docs	3
Total	**100**

9 Submission Instructions

9.1 What to Submit

1. **GitHub Repository URL** containing:
   - All source code following the required structure
   - `requirements.txt` with pinned versions
   - `.env.example` with all required environment variables
   - Complete test suite
   - Docker configuration files
2. **README.md** including:
   - Setup instructions (step-by-step)
   - How to run tests
   - API documentation (or link to Swagger)
   - Design decisions and trade-offs
   - Known limitations
3. **Screenshots** (in `docs/` folder):
   - Health check response
   - Swagger UI showing all endpoints
   - Successful test run output

9.2 Submission Process

1. Push your final code to GitHub
2. Ensure the repository is accessible (public or shared with instructor)
3. Submit the repository URL via the course LMS by **February 6, 3:59 PM**
4. Late submissions: 10% penalty per day, up to 3 days

9.3 Academic Integrity

- This is **individual work**
- You may discuss concepts with classmates but must write your own code
- AI assistants (ChatGPT, Copilot) are permitted for debugging and learning
- You must understand and be able to explain all code you submit
- Cite any external resources or code snippets used

10 Resources

10.1 Documentation

- FastAPI: https://fastapi.tiangolo.com
- Pydantic v2: https://docs.pydantic.dev/latest/
- Snowflake Python Connector: https://docs.snowflake.com/en/developer-guide/python-connector/python-connector
- Redis-py: https://redis-py.readthedocs.io/
- Docker: https://docs.docker.com/

10.2 Starter Repository

Clone the starter repository:

```
git clone https://github.com/quantuniversity/pe-org-air-cs1-starter.git
cd pe-org-air-cs1-starter
```

The starter includes:

- Project structure scaffolding
- Configuration templates
- Test fixtures
- Docker base configuration

10.3 Getting Help

1. Check the course Slack/Discord channel
2. Attend office hours (schedule on LMS)
3. Post questions during lab sessions
4. Review Lab 1 and Lab 2 recordings

11 Looking Ahead

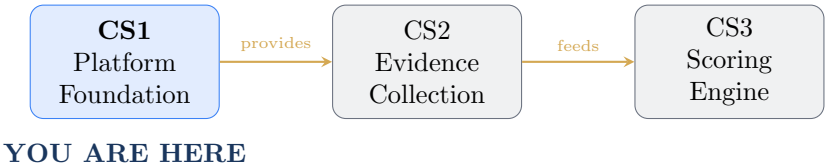

Figure 3: Your Journey Through the Course

What you're building now becomes the foundation for everything else:

- **Case Study 2** will add SEC document ingestion and external signals—stored using your database schema

- **Case Study 3** will implement the scoring engine—using your Pydantic models
- **Case Study 4** will add RAG and semantic search—building on your API
- **Case Study 5** will add dashboards—visualizing data from your platform

"No data? No API? No platform.
We start here."

Good luck! — Sri Krishnamurthy